

Chapter 7

More SQL: Complex Queries, Triggers, Views, and Schema Modification

1

NULL in SQL Queries

- SQL allows queries that check if a value is **NULL**.
- SQL uses **IS** or **IS NOT** to check whether an attribute value is NULL or not, because it considers each NULL value distinct from other NULL values
 - *equality comparison* is not appropriate in that case.
- Query 18: Retrieve the names of all employees who do not have supervisors.

```
Q18:  SELECT  Fname, Lname
      FROM    EMPLOYEE
      WHERE   Super_ssn IS NULL;
```

2

Nesting of Queries

- A **nested query** is a complete SELECT query that is specified within the WHERE clause of another query, called **outer query**.
- Query 1: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1A: SELECT      Fname, Lname, Address
      FROM        EMPLOYEE
      WHERE       Dno IN
                  (SELECT Dnumber
                     FROM   DEPARTMENT
                     WHERE  Dname='Research' );
```

- The nested query selects the Dnumber of the 'Research' department.
- The outer query select an EMPLOYEE tuple if its Dno value is in the result of the nested query.

3

Nesting of Queries (cont'd)

- The comparison operator **IN** compares a value v with a set (or bag) of values V (that are compatible with v), and evaluates to **TRUE** if v is one of the elements in V .
- **NOT IN** is the negation of **IN**.
- In general, we can have several levels of nested queries.
- We can consider that for each tuple (or combination of tuples) of the outer query, the **WHERE clause (condition)** is evaluated.

4

Nesting of Queries (cont'd)

- Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

```
Q4C: SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN
    ((SELECT Pnumber
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber=Pno AND Essn=Ssn AND
      Lname='Smith')
UNION
    (SELECT Pnumber
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum=Dnumber AND Mgr_ssn=Ssn AND
      Lname='Smith'));
```

5

Nesting of Queries (cont'd)

- Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

```
Q4B: SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN
    (SELECT Pnumber
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber=Pno AND Essn=Ssn AND
      Lname='Smith')
OR
    Pnumber IN
    (SELECT Pnumber
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum=Dnumber AND Mgr_ssn=Ssn AND
      Lname='Smith');
```

6

Nesting of Queries (cont'd)

- SQL allows a tuple of values (i.e, a list of values) to be compared with a set of tuples (that are compatible) by using the **IN** operator by placing them within parentheses.

Example: Retrieve the SSN of each employee who works on the same (project, hours) combination that 'John Smith' works on.

```
SELECT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN
      (SELECT Pno, Hours
       FROM EMPLOYEE, WORKS_ON
       WHERE Fname='John' AND Lname='Smith'
              AND Ssn=Essn);
```

Note: Essn of John Smith is also retrieved.

7

Other Comparison Operators

- In addition to **IN** operator, other comparison operators (**=**, **>**, **>=**, **<**, **<=**, and **<>**) can be used to compare a single value *v* with a set (or bag) of values *V*.
- A keyword **ANY** (or **SOME**) or **ALL** can be used to specify any (or some) or all of the values, respectively, in a set (or a bag) of values *V* for the comparison.
- Example: Retrieve the last name and first name of employees whose salary is greater than the salary of all the employees in department 5.

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL (SELECT Salary
                   FROM EMPLOYEE
                   WHERE Dno=5 );
```

8

Other Comparison Operators (cont'd)

- Example: Retrieve the last name and first name of employees whose salary is the same as the salary of any employee in department 5.

```

SELECT  Lname, Fname
FROM    EMPLOYEE
WHERE   Salary = ANY (SELECT Salary
                       FROM    EMPLOYEE
                       WHERE   Dno=5 );

```

- IN is same as = ANY

```

SELECT  Lname, Fname
FROM    EMPLOYEE
WHERE   Salary IN (SELECT Salary
                   FROM    EMPLOYEE
                   WHERE   Dno=5 );

```

9

Correlated Nested Queries

- If a condition in the WHERE clause of a nested query references an attribute of a relation declared in the (FROM clause of) outer query, the two queries are said to be *correlated*.
 - The result of a correlated nested query may be different for each tuple (or combination of tuples) of the relation(s) declared in the outer query
- Query 16: Retrieve the name of each employee who has a dependent with the same first name and same sex as the employee.

```

Q16: SELECT  E.Fname, E.Lname
      FROM    EMPLOYEE AS E
      WHERE   E.Ssn IN
              (SELECT  Essn
               FROM    DEPENDENT AS D
               WHERE   E.Fname=D.Dependent_name
                       AND E.Sex=D.Sex);

```

10

Uncorrelated Nested Query

- Query 16: Retrieve the name of each employee who has a dependent with the same first name and same sex as the employee.

• Q16B: **SELECT** E.Fname, E.Lname
FROM EMPLOYEE **AS** E
WHERE E.Ssn **IN** (**SELECT** D.Essn
FROM DEPENDENT **AS** D,
EMPLOYEE **AS** F
WHERE D.Essn=F.Ssn **AND**
F.Fname=D.Dependent_name
AND F.Sex=D.Sex);

11

Correlated Nested Queries (cont'd)

- A query having a nested select-from-where block and using **IN** or '**= ANY**' operator can always be expressed as a single block query.

For example, Q16 may be written as Q16A.

Q16A: **SELECT** E.Fname, E.Lname
FROM EMPLOYEE **AS** E, DEPENDENT **AS** D
WHERE E.Ssn=D.Essn **AND** E.Sex=D.Sex
AND E.Fname=D.Dependent_name;

12

EXISTS Function

- **EXISTS** is used to check whether the result of a correlated nested query contains a tuple or empty (i.e., contains no tuples).
 - EXISTS function takes a correlated nested query as an argument and returns TRUE if the result of nested query contains at least one tuple; otherwise, it returns FALSE.
- **NOT EXISTS** is the negation of EXISTS.
- **EXISTS is more flexible than IN** because IN operator compares a value with a set of values, both of which have the same data type.

13

EXISTS Function (cont'd)

- Query 16: Retrieve the name of each employee who has a dependent with the same first name and same sex as the employee.

```
Q16B:  SELECT  Fname, Lname
        FROM    EMPLOYEE AS E
        WHERE   EXISTS
                (SELECT  *
                 FROM    DEPENDENT AS D
                 WHERE   E.Ssn=D.Essn AND
                        E.Fname=D.Dependent_name
                        AND E.Sex=D.Sex);
```

14

EXISTS Function (cont'd)

- Query 6: Retrieve the names of employees who have no dependents.

```
Q6: SELECT Fname, Lname
      FROM EMPLOYEE AS E
      WHERE NOT EXISTS
            (SELECT *
             FROM DEPENDENT AS D
             WHERE E.Ssn=D.Essn);
```

- In Q6, the correlated nested query retrieves all DEPENDENT tuples related to the EMPLOYEE tuple being evaluated. If none exists, the EMPLOYEE tuple is selected.

15

EXISTS Function (cont'd)

- Query 6: Retrieve the names of employees who have no dependents.

```
Q6: SELECT Fname, Lname
      FROM EMPLOYEE AS E
      WHERE NOT EXISTS
            (SELECT *
             FROM DEPENDENT AS D, EMPLOYEE AS F
             WHERE F.Ssn=D.Essn AND F.Ssn=E.Ssn);
```

- If EMPLOYEE relation is included in the FROM clause of the nested query, then it is an error because the nested query is not correlated with the outer query. In that case, 'Ssn=Ssn' should be added in the where clause of the nested query, in order to correlate the nested query to the current EMPLOYEE tuple being evaluated in the outer query.

16

EXISTS Function (cont'd)

- Query 3: Retrieve the name of each employee who works on all the projects controlled by department 5.

```
Q3A: SELECT  Fname, Lname
      FROM    EMPLOYEE AS E
      WHERE    NOT EXISTS
                ((SELECT  Pnumber
                   FROM    PROJECT AS P
                   WHERE    P.Dnum=5)
                 EXCEPT (SELECT W.Pno
                           FROM    WORKS_ON AS W
                           WHERE    E.Ssn=W.Essn));
```

- EXCEPT** (or **MINUS** in some systems) is the set difference operation.

17

EXISTS Function (cont'd)

- Query 3: Retrieve the name of each employee who works on all the projects controlled by department 5.

```
Q3C: SELECT  Fname, Lname
      FROM    EMPLOYEE AS E
      WHERE    NOT EXISTS
                (SELECT  *
                 FROM    PROJECT AS P
                 WHERE    P.Dnum=5 AND
                        P.Pnumber NOT IN
                        (SELECT W.Pno
                         FROM    WORKS_ON AS W
                         WHERE    E.Ssn=W.Essn));
```

18

Explicit Set of Values in the WHERE clause

- IN operator can compare an attribute value with an explicit set of values in the WHERE clause.
- Query 17: Retrieve the social security numbers of all employees who work on project number 1, 2, or 3.

Q17: **SELECT DISTINCT** Essn
 FROM WORKS_ON
 WHERE Pno **IN** (1, 2, 3);

19

Joined Relations in SQL-92

- A join condition can be moved from the WHERE clause to the FROM clause by using keywords JOIN and ON, like

FROM (EMPLOYEE **JOIN** DEPARTMENT
 ON Dno=Dnumber)

- The result of a join (i.e., a relation created by a join) in the FROM clause looks like an input relation, and it is called a "joined relation."
- Allows the user to specify different types of joins: **JOIN** (in some systems, **INNER JOIN**), **NATURAL JOIN**, **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, **FULL OUTER JOIN**.

20

Joined Relations in SQL-92 (cont'd)

Q1: **SELECT** Fname, Lname, Address
FROM EMPLOYEE, DEPARTMENT
WHERE Dname='Research' **AND**
Dno=Dnumber;

- Q1 can be written as:

Q1A: **SELECT** Fname, Lname, Address
FROM (EMPLOYEE **JOIN** DEPARTMENT
ON Dno=Dnumber)
WHERE Dname='Research';

21

Joined Relations in SQL-92 (cont'd)

- It is possible to nest join specifications; that is, a join output is joined with a table, and it is called a *multiway join*.
- Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and that department manager's last name, address, and birthdate.

Q2A: **SELECT** Pnumber, Dnum, Lname, Address, Bdate
FROM ((PROJECT **JOIN** DEPARTMENT **ON**
Dnum=Dnumber) **JOIN** EMPLOYEE
ON Mgr_ssn=Ssn)
WHERE Plocation='Stafford';

22

Joined Relations in SQL-92 (cont'd)

- Query 8: For each employee, retrieve the employee's last name and the last name of his/her immediate supervisor (if exists).

```
Q8B: SELECT E.Lame AS Employee_Lname,  
        S.Lname AS Supervisor_Lname  
      FROM (EMPLOYEE AS E  
            LEFT OUTER JOIN  
            EMPLOYEE AS S  
            ON E.Super_ssn=S.Ssn);
```

23

Aggregate Functions

- Include **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**.
- **COUNT** function counts the tuples or non-Null values of an attribute.
- Query 19: Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```
Q19: SELECT SUM (Salary), MAX (Salary),  
        MIN (Salary), AVG (Salary)  
      FROM EMPLOYEE;
```

24

Aggregate Functions (cont'd)

- Queries 21: Retrieve the total number of employees in the company.

Q21: **SELECT** **COUNT (*)**
 FROM EMPLOYEE;

- Query 22: Retrieve the number of employees in the 'Research' department.

Q22: **SELECT** **COUNT (*)**
 FROM EMPLOYEE, DEPARTMENT
 WHERE Dno=Dnumber **AND** Dname='Research';

- Query 23: Count the number of distinct salary values of employees.

Q23: **SELECT** **COUNT (DISTINCT Salary)**
 FROM EMPLOYEE;

25

Aggregate Functions (cont'd)

- We can specify a correlated nested query with an aggregate function.
- In SQL, as the result of each aggregate function is a value, it can be compared with a numeric value or a numeric attribute value.
- Query 5: Retrieve the names of all the employees who have two or more dependents.

Q5: **SELECT** Lname, Fname
 FROM EMPLOYEE
 WHERE (**SELECT COUNT (*)**
 FROM DEPENDENT
 WHERE Ssn=Essn) **>= 2**;

26

Grouping

- In many cases, we want to apply the aggregate functions to the groups of tuples in a relation.
- Each group of tuples consists of the tuples that have the *same value(s)* for the *grouping attribute(s)*.
- The aggregate function is applied to each group independently.
- SQL has a **GROUP BY** clause for specifying the grouping attributes, which must also appear in the **SELECT** clause (together with aggregate functions).

27

Grouping (cont'd.)

- Query 24: For each department, retrieve the department number, the number of employees in the department, and their average salary.

Q24: **SELECT** Dno, **COUNT** (*), **AVG** (Salary)
FROM EMPLOYEE
GROUP BY Dno;

- When the **GROUP BY** clause is used, the **SELECT** clause includes only the grouping attribute and the aggregate functions to be applied to each group of tuples.

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

Grouping EMPLOYEE tuples by the value of Dno

Grouping (cont'd.)

- If there is a **WHERE** clause, it is applied to the tuples first, then only the qualified tuples are grouped.
- Query: For each department, retrieve the department number, the number of employees whose (individual) salary is higher than 2000 in the department, and their average salary.

```
SELECT    Dno, COUNT (*), AVG (Salary)
FROM      EMPLOYEE
WHERE      Salary > 20000
GROUP BY  Dno;
```

29

Grouping (cont'd)

- Query 25: For each project, retrieve the project number, the **project name**, and the number of employees who work on that project.

```
Q25: SELECT Pnumber, Pname, COUNT (*)
FROM      PROJECT, WORKS_ON
WHERE      Pnumber=Pno
GROUP BY  Pnumber, Pname;
```

- In this case, the grouping and aggregate functions are applied after the two relations are joined.
- Pname is included in the **GROUP BY** clause, even though Pnumber is enough to group the tuples, because the **SELECT** clause can include only grouping attributes and aggregate functions (when the **GROUP BY** clause is used).

30

HAVING clause

- Sometimes we want to retrieve the values of aggregate functions for only those groups that satisfy certain condition(s).
- The **HAVING** clause is used for specifying a selection condition on groups (rather than on individual tuples).

31

HAVING clause (cont'd)

- Query 26: For each project **on which more than two employees work**, retrieve the project number, the project name, and the number of employees who work on that project.

```
Q26:  SELECT Pnumber, Pname, COUNT (*)
        FROM PROJECT, WORKS_ON
        WHERE Pnumber=Pno
        GROUP BY Pnumber, Pname
        HAVING COUNT (*) > 2;
```

32

Summary of SQL Queries

- A query in SQL can consist of up to six clauses, but only the first two — SELECT and FROM clauses — are mandatory. The clauses are specified in the following order:

SELECT <attribute and function list>
FROM <table list>
[WHERE <condition>]
[GROUP BY <grouping attribute(s)>]
[HAVING <group condition>]
[ORDER BY <attribute list>;

33

Assertions

- An assertion is used to specify a general constraint that cannot be specified by the basic constraints of the relational model (i.e., key constraint, entity integrity constraint, referential integrity constraint, and domain constraint).
- An assertion specifies a query to check if there is a case (i.e., tuple) violating the constraint.
 - The query result must be empty to satisfy the constraint represented by the assertion.
 - If the query result is not empty, the constraint represented by the assertion has been violated.

34

Assertion Example

- Constraint: The salary of each employee should not be greater than the salary of the manager of the department the employee works for.

```
CREATE ASSERTION SALARY_CONSTRAINT  
CHECK (NOT EXISTS  
  (SELECT *  
    FROM EMPLOYEE AS E,  
          EMPLOYEE AS M,  
          DEPARTMENT AS D  
    WHERE E.Dno=D.Dnumber AND  
          D.Mgr_ssn=M.Ssn AND  
          E.Salary > M.Salary  
  )  
);
```

35

SQL Triggers

- Objective: To monitor a database and initiate an action (by invoking a *stored procedure* or a *user defined function*) when a certain condition occurs.
- Triggers are expressed in a syntax similar to assertions and include the following:
 - Event
 - Such as an INSERT, DELETE, or UPDATE operation
 - Condition (violating the constraint)
 - Action
 - To be taken when the condition is true.

36

An Example Trigger

- Constraint: The salary of each employee should not be greater than the salary his/her supervisor.
- A trigger to compare an employee's salary to his/her supervisor when the INSERT or UPDATE operation is (or to be) executed:

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF
Salary, Super_ssn ON EMPLOYEE
FOR EACH ROW
WHEN (NEW.Salary > (SELECT Salary
                        FROM EMPLOYEE
                        WHERE Ssn=NEW.Super_ssn))
INFORM_SUPERVISOR (NEW.Super_ssn, NEW.Ssn);
```

37

Views in SQL

- A view is a virtual, named table that is derived from other (base and virtual) tables.
 - A base table means a relation (i.e., a state of a relation schema).
- Allows full retrieval operations on views.
- Allows limited update operations on views since the updated values may not be physically stored in the database.

38

Specification of a View

- SQL command: **CREATE VIEW** contains
 - a view name.
 - a possible list of attribute names. For example, when arithmetic operations are used or when we want the names to be different from the names of the attributes in the underlying tables.
 - a query to specify the content of the view (i.e., a query whose output would be the content of the view).

39

Example Views

WORKS_ON1

Fname	Lname	Pname	Hours
-------	-------	-------	-------

DEPT_INFO

Dept_name	No_of_emps	Total_sal
-----------	------------	-----------

- V1: **CREATE VIEW** WORKS_ON1
AS SELECT Fname, Lname, Pname, Hours
FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE Ssn=Essn **AND** Pno=Pnumber;
- V2: **CREATE VIEW** DEPT_INFO (Dept_name, No_of_emps, Total_sal)
AS SELECT Dname, **COUNT**(*), **SUM**(Salary)
FROM DEPARTMENT, EMPLOYEE
WHERE Dnumber=Dno
GROUP BY Dname;

40

Using Views

- We can specify SQL queries on a created view:

```
QV1: SELECT Fname, Lname  
FROM WORKS_ON1  
WHERE Pname='ProductX';
```

- If a view is no longer needed, it can be dropped:

```
DROP VIEW WORKS_ON1;
```

41

Update Views

- A single view created from a base table without using an aggregate function is updatable.
 - As the update can be mapped to an update on the underlying base table.
- Views defined using groups and an aggregate function are not updatable.
- Views defined on multiple tables using joins are generally not updatable, because the interpretation of the update query may not be clear.

```
UV1: UPDATE WORKS_ON1  
SET Pname = 'ProductY'  
WHERE Lname='Smith' AND Fname='John'  
AND Pname='ProductX';
```

Problem: UV1 can be interpreted as: (1) change the Pname from 'ProductX' to 'ProductY' (on which John Smith is working), or (2) change the project assignment of John Smith from 'ProductX' to 'ProductY'.

42